

Now that your Conda environment is configured and VSCode is installed, we can start to bring everything together.

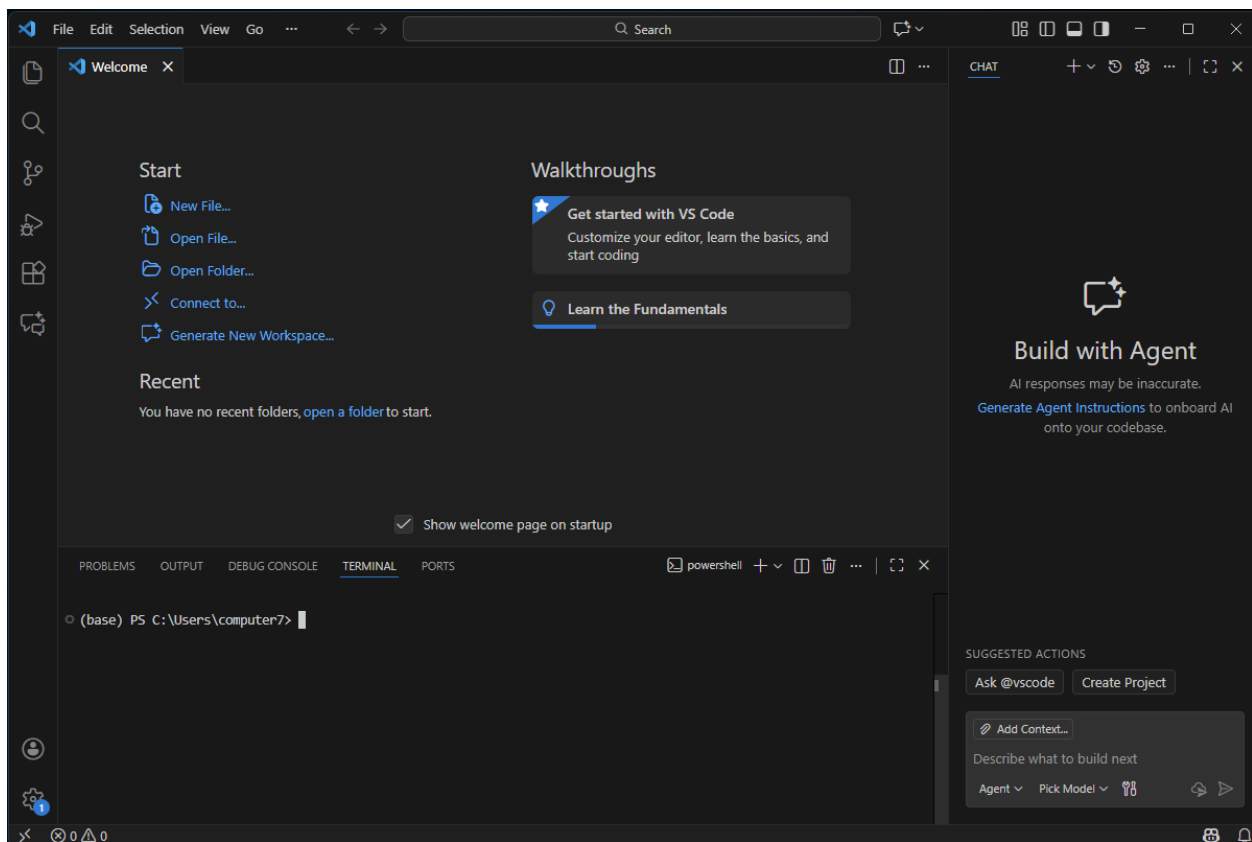
First, open the Anaconda PowerShell Prompt and execute the following commands*:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser  
  
conda init powershell
```

*Note: For those concerned about security, here we are setting the current user's execution policy to "remote signed" which allows your computer to run external scripts as long as you initialize this through the command line (i.e. by typing `conda init powershell`). You may reset your execution policy after initializing Conda.

This should ensure that VSCode recognizes your Miniconda installation through your computer's native PowerShell terminal; this allows you to configure your Conda environment through the command line entirely within VSCode without having to open external terminals!

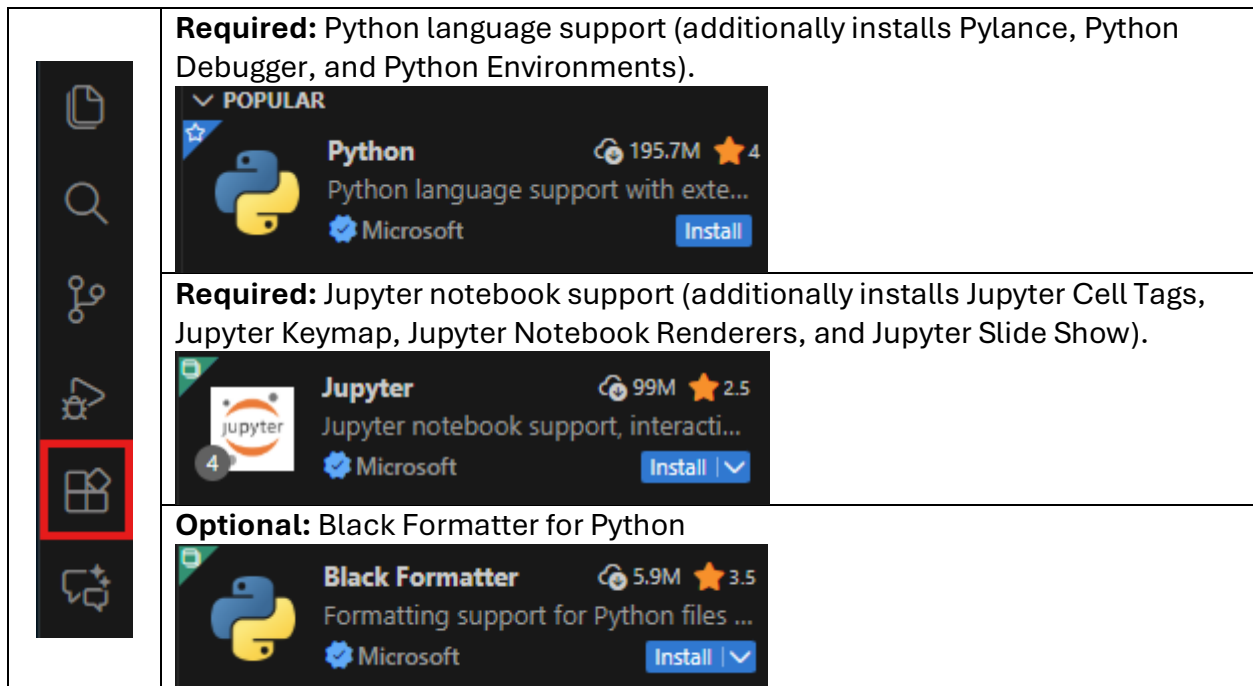
Next, we can close the Anaconda PowerShell Prompt and open VSCode. If configured correctly, pressing `Ctrl+`` or Selecting "Terminal > New Terminal" from the top-left task bar should display a PowerShell terminal in VSCode like so:



If (base) is displayed before the input line, you can now activate/configure your Conda environment directly through VSCode!

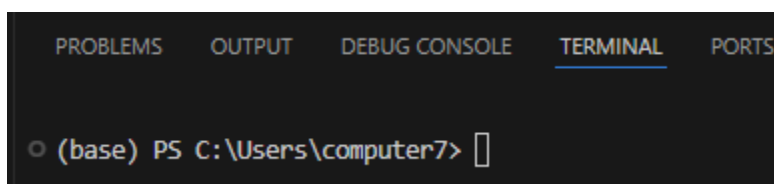
VSCode is an extensive development interface that supports the vast majority of modern programming languages, but for this workshop, we will only need to install the VSCode-specific extensions that support Python and Jupyter while making them easier to use.

Pressing Ctrl+Shift+X or clicking on the “Extensions” tab within the Activity Bar (located on the lefthand side of the window by default) allows us to search for the extensions we need:



The Python Black Formatter is an optional extension that helps you adhere to conventional Python standards so that your code is easier for other developers to read, follow, and potentially collaborate with in the future.

Now that VSCode is set up to recognize and edit Python, we can make a directory to store the work we’ll be doing. Your terminal should display that you are in your home directory which should look like this:



Where your username should be where you see “computer7”

From here, we can create a dedicated directory for this workshop. I'm going to name it "commlab-datavis-workshop-2026", but you can name it anything you'd like! We can use the following command to perform this:

```
mkdir commlab-datavis-workshop-2026
```

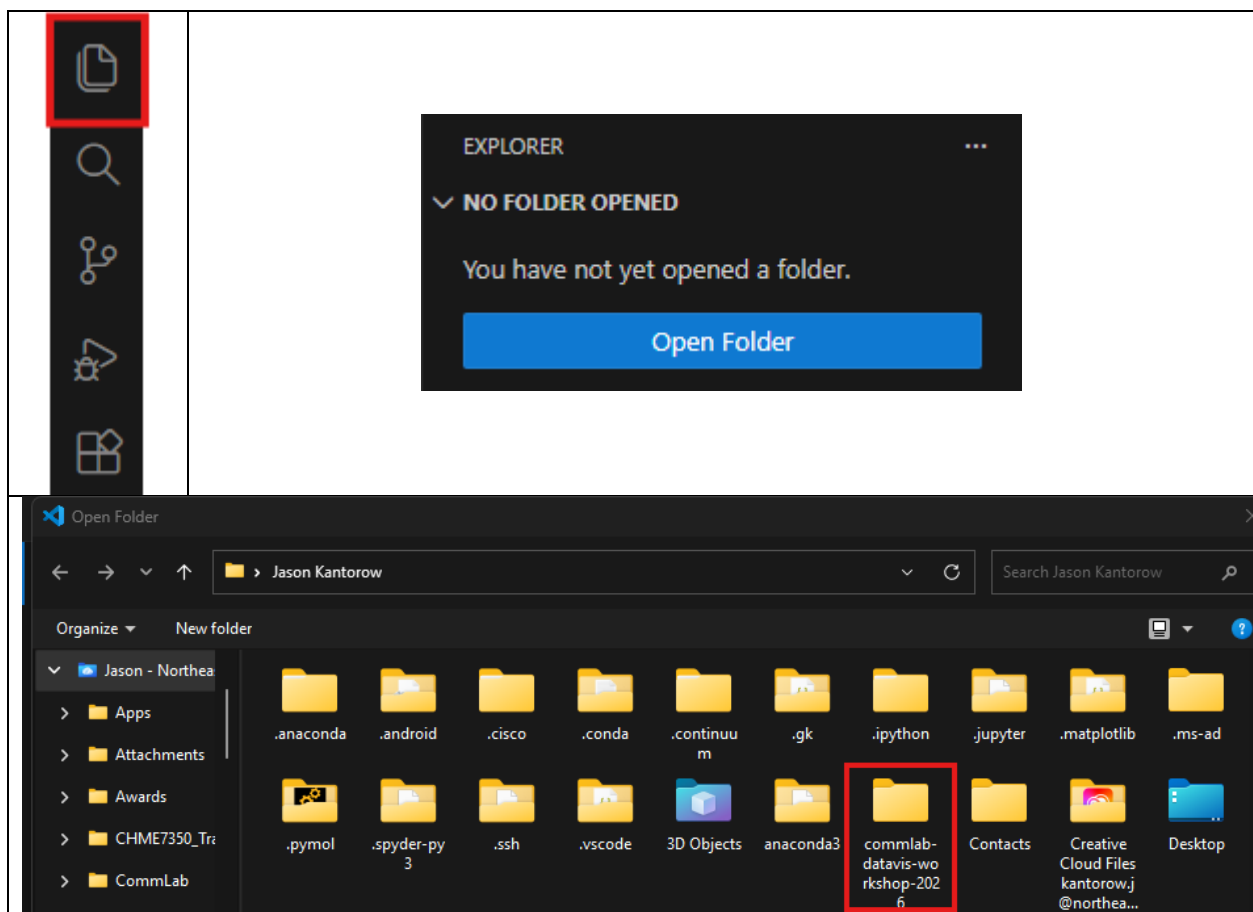
We can also change our current working directory to this newly created one using the following command:

```
cd .\commlab-datavis-workshop-2026\
```

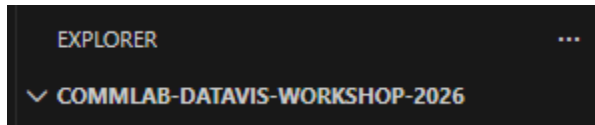
Your terminal input line should now look something like this:

```
(base) PS C:\Users\computer7\commlab-datavis-workshop-2026>
```

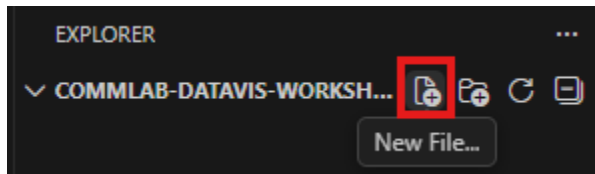
Pressing Ctrl+Shift+E or clicking on the "Explorer" icon in the Activity Bar allows us to select a folder where we'll be working. This should bring up your home directory by default where we can select the folder we just made. If your home directory is not displayed by default, you can copy the path "C:\Users\yourusername\commlab-datavis-workshop-2026" into the search bar at the top of the "Open Folder" window.



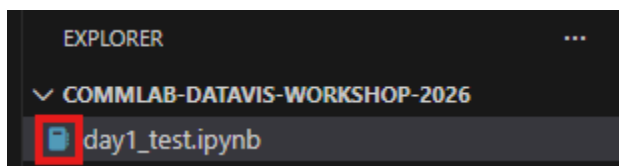
Clicking “Select Folder” should display the following in the Explorer window:



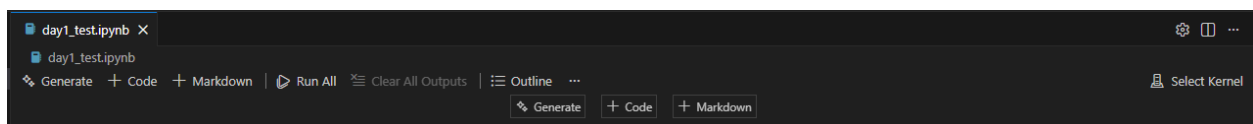
From here, we can create our first Jupyter Notebook file by hovering over the folder name and clicking the “New File...” icon:



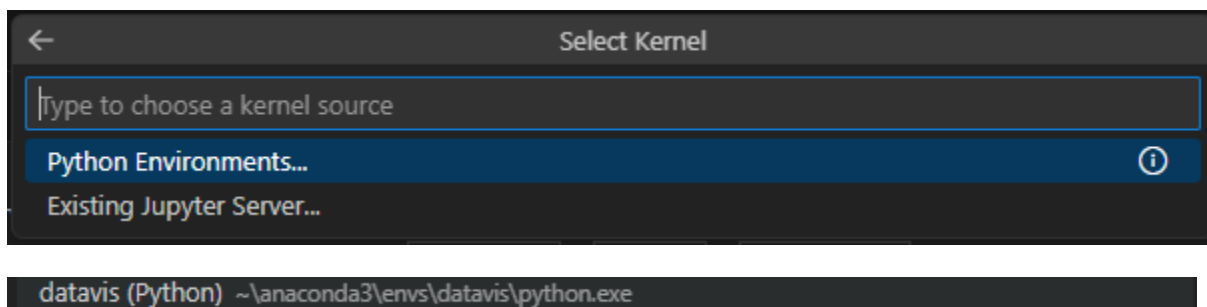
VSCode will then bring up a text box in which to name your file. I will call it “day1_test.ipynb” but you can name it whatever you’d like as long as you include the extension “.ipynb” so that VSCode can recognize that it is a Jupyter Notebook file.



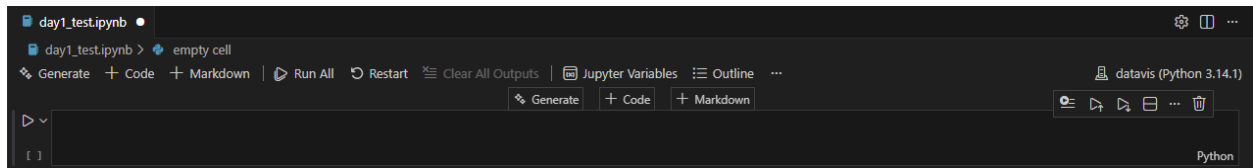
VSCode should include a notebook-looking icon before the file name ensuring that it recognizes the extension. Pressing Enter should bring up a window in the middle workspace displaying a blank notebook:



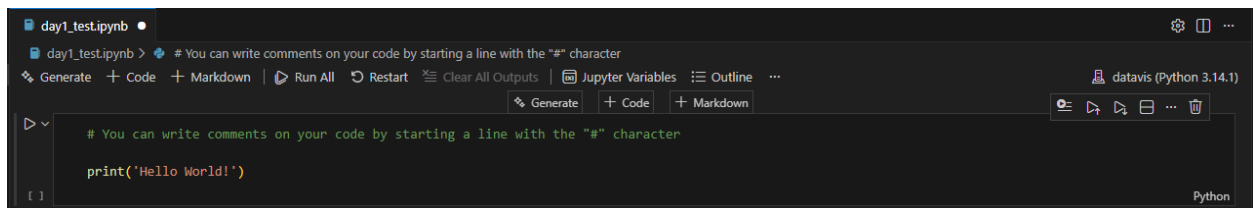
Clicking on “Select Kernel” in the upper right allows us to specify the Python environment we’ll be working with:



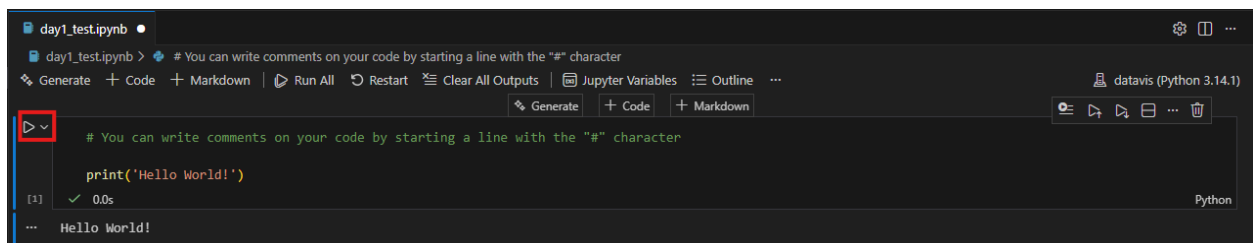
With the kernel set up, we can create our first code cell with the “+ Code” button at the top middle of the screen. This is the main method we’ll be using to write and execute Python code:



In the code cell, we can write our first simple program and test the output. We can use the Python `print()` function to print a message to the output. Note that the message is in quotes; this creates a type of object in Python called a “string” which contains ascii characters.



You can execute this cell by pressing “Ctrl+Enter” or clicking the “Execute Cell” icon resembling a triangle at the top left of the cell:



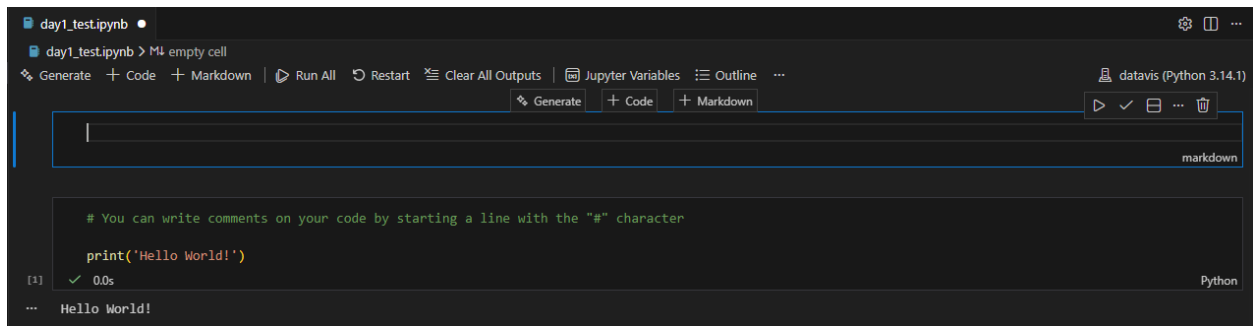
The output of this cell should appear directly below it. Notice the three new icons that have appeared upon execution:



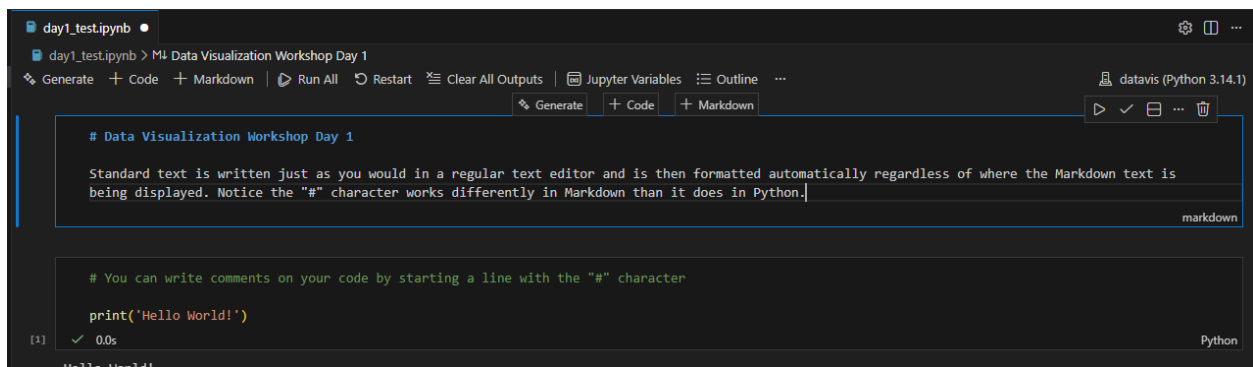
The number in brackets indicates the order in which you have executed each cell so far starting from one. The check mark indicates that all the code in this cell has executed successfully without error; a red “X” would appear along with an error message in the output otherwise. The 0.0s indicates the time taken to completely execute all code within this cell.

In addition to Python code, we can create and use Markdown cells to insert standard text and easily format it into headers, bullet points, tables, and more. Clicking the “+

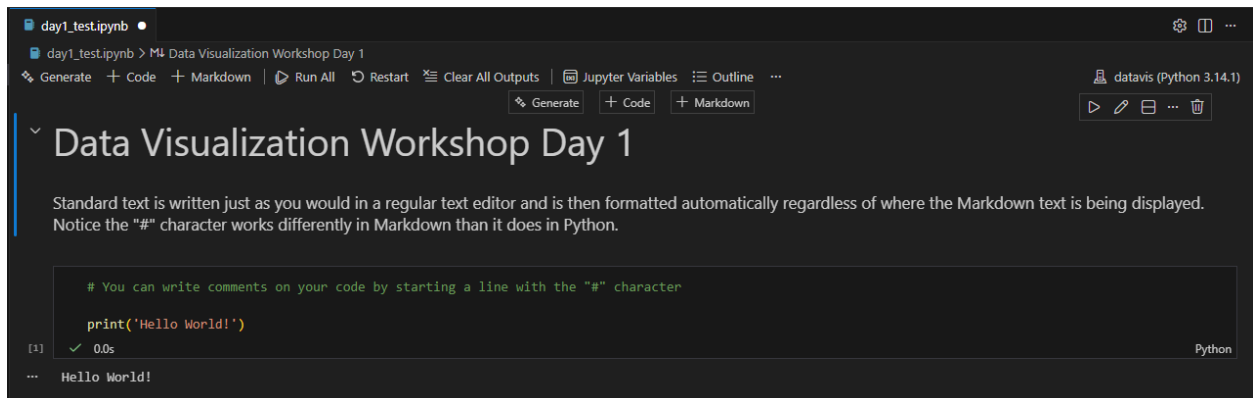
Markdown” button at the top center of the notebook will create a Markdown cell right before our code cell:



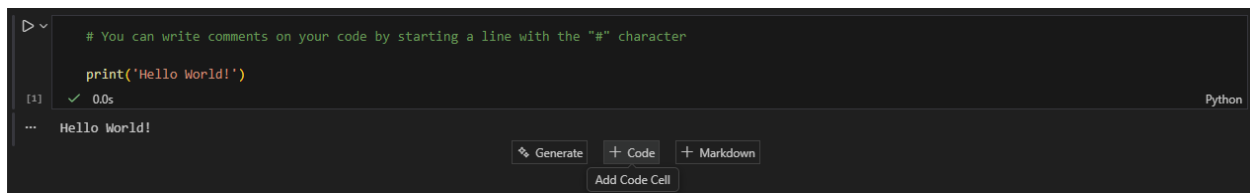
We can create a title by typing a “#” sign and a space before the title text:



Clicking the check mark icon displays the Markdown text fully formatted:



Lastly, we can ensure that our datavis environment packages are accessible here by importing them in a cell below the first code cell. Hovering over the space under the output of the first code cell, click the “+ Code” button to make a new code cell:



A screenshot of a Jupyter Notebook cell. The cell contains a comment line starting with a hash symbol and a Python print statement. The output of the cell is 'Hello World!'. The cell is labeled [1] and has a green checkmark indicating successful execution. The language is set to Python. Below the cell are buttons for 'Generate', '+ Code', '+ Markdown', and 'Add Code Cell'.

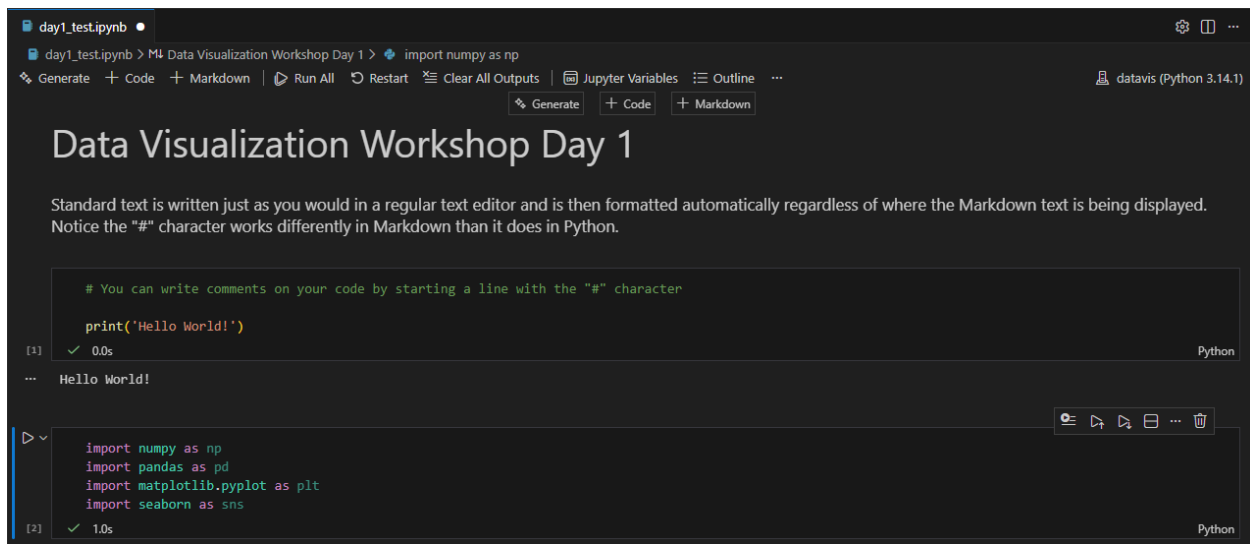
```
# You can write comments on your code by starting a line with the "#" character  
print('Hello World!')
```

[1] ✓ 0.0s Python

... Hello World!

Generate + Code + Markdown Add Code Cell

In a new cell, we can use the Python import statement to bring our required external packages into the scope of our current workspace:



A screenshot of a Jupyter Notebook interface. The top part shows a cell with a 'Hello World' example. Below it is a new cell with import statements for numpy, pandas, matplotlib, and seaborn. The new cell is labeled [2] and has a green checkmark. The language is set to Python. The interface includes a toolbar with buttons for 'Generate', '+ Code', '+ Markdown', 'Run All', 'Restart', 'Clear All Outputs', 'Jupyter Variables', 'Outline', and a file icon. The notebook title is 'day1_test.ipynb' and the kernel is 'datavis (Python 3.14.1)'.

Data Visualization Workshop Day 1

Standard text is written just as you would in a regular text editor and is then formatted automatically regardless of where the Markdown text is being displayed. Notice the "#" character works differently in Markdown than it does in Python.

```
# You can write comments on your code by starting a line with the "#" character  
print('Hello World!')
```

[1] ✓ 0.0s Python

... Hello World!

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns
```

[2] ✓ 1.0s Python

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline ... datavis (Python 3.14.1)

The check mark in the bottom left of this new cell tells us all packages are being successfully recognized!